ICT374 Lab 2:
Unix Shells, Standard I/O Redirections, Archiving and Compressions; C's Formatted Input, and Obtaining Command Line Arguments.

Learning Objectives:

1. Learn to use Unix shells, especially bash shell
2. Learn to do standard I/O redirections.
3. Learn to use command `tar` to archive files and to extract files from archives
4. Learn to use command `gzip` and `gunzip` to compress and uncompress files
5. Understand the evolution of operating systems
6. Learn to use standard I/O functions for formatted output: `printf`, `fprintf`, and `sprintf`
7. Learn to use standard I/O functions for formatted input: `scanf`, `fscanf`, and `sscanf`.
8. Learn to use command line arguments from C programs.

Required Reading:

- Lecture Notes for Topic 2
- Stallings: Chapter 2
- Rute: Ch 4.18, Ch 7.1, Ch 12, Ch 20.8
- A mini manual for vi editor

Access to Software:

Face-to-Face students using the lab computers should copy your work from the lab computer to a USB drive, or to a online drive such as Microsoft OneDrive or Google Drive. This is necessary as you cannot retain anything on any one of the lab computers.

For Online Flexible students, if you have already installed Linux on your own computer, then use your own computer. Otherwise, you may use the university's Linux server (`eris.ad.murdoch.edu.au`) temporarily until you have installed the Linux on your computer. More information on how to install Linux is available from Unit Resources page.

Unix Exercises

The following exercises are designed to get you familiar with Unix (Linux) systems. More information about these basic Unix commands is available from the relevant chapters in Rute. See the Required Readings.

1. **Unix Shells**

   When you start up a terminal program, Unix runs a shell program automatically inside the terminal. The default shell for most Linux distributions is bash shell (/bin/bash). You can confirm this by typing the command `ps` to see the list of processes that are running on your login session, which should include bash.

   You can also start and stop a shell manually. For example, typing the command `bash` would

start another bash shell process. You can exit from the shell by typing the command `exit` at its prompt.

Now try to start a new shell. For example, bash or tcsh. Type the command `ps -H` to confirm that different (instances of) shells are running. Exit the shell by typing command `exit`.

A shell is a command interpreter. It prints a shell prompt on your terminal, inviting you to enter a command line. It then tries to locate the command and once located, creates a process to run that command line.

Note usually you do not need to manually start a shell. A shell is always run when you start a terminal application.

2. **Shell Scripts**

   Each Unix shell is like a programming language. You can write a shell script and run the shell script in a shell. Many Unix system administration utilities are written as bash shell scripts. Create the file like this

   ```
   #!/bin/bash

   echo my user name is $USER
   echo my home direcory is ~
   echo and it contains these files:
   ls -l ~
   ```

   Note `#!/bin/bash` must be the first line and character "#" must be the first character in the first line.

   Save the above in a file such `listing`. Then run the shell script by typing the command `bash listing`.

3. **Shell Variables**

   Like any programming language, a shell supports variables. One of them is the `PATH` variable. `PATH` variable consists of the search paths the shell uses to locate commands. You can see your current search paths by typing the command `echo $PATH`. Try it.

   In last week's exercises, you run programs, such as `a.out`, with a relative path such as `./a.out`. If you attempt to run the program with just the program name `a.out`, the shell may complain: `Command not found`. This is because the directory containing `a.out` may not be in the search paths. To add your current directory to the search paths, you can issue the following command:

   ```
   %  PATH=$PATH:.
   ```

   Note in the above, like in Lab 1, character "%" denote your shell prompt.

   Try the above command at the shell prompt, then type the command `echo $PATH` again. You will find that the current directory "." is now in the search path. Now you should be able to run program `a.out` by entering only the program name `a.out` if it is in the current directory.

   Type the following command to see a list of variables and their values in your shell process:

   ```
   %  declare  |  more
   ```

4. **Standard I/O Redirections**

Last week, we mentioned standard input, standard output and standard error. To a Unix program, `stdin`, `stdout` and `stderr` are treated as three open files. Usually a shell associates `stdin` with the terminal keyboard, `stdout` and `stderr` with the terminal screen. However such associations can be changed (redirected) to files. For example, the program `ls` sends its output to standard output, ie the terminal screen. However if you redirect the standard output to a file, then output of `ls` would go to that file.

In a shell, we use symbol "<" to redirect standard input, and ">" to redirect standard output, and "2>" to redirect standard error. The following example sends the output from program `ls` to file `foo`:

```
%  ls -l >  foo
```

Type the above command, and then check the content of file `foo` with command `cat foo`.

Our next example shows that the command `sort` reads lines from file `foo` and sort them in alphabetical order and displays the results in the standard output (terminal screen):

```
%   sort <  foo
```

5. **Archive and Extract Files with `tar`**

   We often need to archive files and extract files from an archive. The Unix command for this task is `tar`. The command `tar` stands for Tape ARchive, since in the old days archives were stored on magnetic tapes. The command has numerous options. Check its manual page with `man tar`.

   One commonly used option is to create an archive file:

   ```
   tar -cvf your_tar_filename.tar  directory_name
   ```

   to create a tar file *`your_tar_filename.tar`* from files in the directory *`directory_name`*. For example the following command:

   ```
   %   tar  -cvf   ~/lab02.tar  ~/lab02
   ```

   The above command would create tar file `lab02.tar` containing all files under directory `lab02`. The tar file is stored under your home directory.

   Another commonly used option is to extract all files from tar file (often called tar ball):

   ```
   %  mkdir tmp
   %  cd tmp
   %  tar -xvf ~/lab02.tar
   ```

   The first command creates a new directory `tmp`. The second command sets your current directory to `tmp`. The last command extracts all files in directory `tmp`.

   Try the above examples. Try to create archives from other directories and extract the tar files (in a different directory).

6. **File Compression and Decompression with `gzip` and `gunzip`**

   To compress a file, say `lab02.tar`, use command

   ```
   %  gzip  lab02.tar
   ```

this will create a compressed file named `lab02.tar.gz`. To uncompress it, just type the command

```
%   gunzip lab02.tar.gz
```

Try the two commands on a file

Note that on Linux system, compression and uncompression are added to the tar command. For example, you can archive files and then compress the archive in a single command using option `z`, such as

```
%   tar  -zcvf   ~/lab02.tar.gz  ~/lab02
```

Theory Exercises

1. Explain why an operating system is said to be an interface between hardware and applications.
2. What is the kernel of an operating system?
3. What is multiprogramming? Why do we need it?
4. What is a process? What are the three components of a process?
5. What is the virtual address and what is the relationship between a virtual address and its physical address?
6. Why conversion from a virtual address to its physical address is normally done by the hardware?

C Programming Exercises

Note, all exercises must be done via the terminal commands, not GUI tools. Do not use IDE to build your programs. Do not use GUI file managers to manage your directories and files. Also each exercise should be in a separate directory, which itself, is under a directory called `lab02`

1. **Get Input From Standard Input with `scanf`**

   Assume we want to receive user input of date, month and year in the form of *`dd/mm/yyyy`* such as `08/08/2006`, is there an easy way to get the date, month and year into three separate variables? The standard function `scanf` can help. The following code prompts the user to enter the date in the form of "dd/mm/yyyy" and reads the date, the month, and the year into variables `date`, `month` and `year`:

   ```
   int date, month, year;

   printf("Please enter the date in the form of dd/mm/yyyy: ");
   scanf("%d/%d/%d", &date, &month, &year);

   printf("the date you entered was:  %d-%d-%d\n", date, month, year);
   ```

   Note you must pass the addresses of variables `date` (ie, `&date`), `month` (ie, `&month`) and `year` (ie, `&year`), not their values, to function `scanf`.

   Convert the above code into a complete program. Test the program with some keyboard input of dates.

2. **`scanf` Again**

   Assume the input is in the form of date-month-year, such as 8-August-2006, how would you read the date, the month and the year? Note in this case, the month is not a number. It is a string. See the following code:

```
int date;
char month[BUF_SIZE];
int year;

printf("Please enter the date in the form of date-short_month-year, ");
printf(" such as 8-Aug-2006, where a month is written in exactly three letters: ");
scanf("%d-%3s-%d", &date, month, &year);
```

   Please note in `scanf`, we pass the address of array `month` without using the address operator &. This is because the array name **`month`** itself represents the starting address of the array.

   Convert the above code into a complete program and test it with date inputs.

3. **Get Input from a File with `fscanf`**

   As with `printf` and `fprintf`, there is also a function similar to `scanf` that allows you to get input directly from a file rather than just from the standard input. It is called `fscanf`. See the following code:

```
char *fname="foo3";
FILE *fstr;
int date;
char month[BUF_SIZE];
int year;

fstr = fopen(fname, "r");
if (fstr == NULL) {
    fprintf(stderr, "Cannot open file %s, exit.\n", fname);
    exit(1);
}

fscanf(fstr, "%d-%3s-%d", &date, month, &year);
printf("date = %d, month = %s, year = %d\n", date, month, year);
```

   Please note that you should close the file `fstr` as soon as you have finished using it with `fclose(fstr)`.

   Convert the above code into a complete program. Create a text file containing that date of format "08-Aug-2006" and test the program using the date file.

4. **Get Stuff From String Using `sscanf`**

   The following code demonstrates how to extract information from a string with the standard function `sscanf`:

```
char *date_str = "8-Aug-2006";
int date;
char month[BUF_SIZE];
int year;

sscanf(date_str, "%d-%3s-%d", &date, month, &year);
printf("Date:  %s %d, %d\n", month, date, year);
```

   Convert the above code into a complete program and test it with different date strings.

5. **Get Command Line Arguments**

   We often run a command with additional arguments from the command line, such as

   ```
   ls -l -t  abc xyz bla
   ```

   where `ls` is the command name, or command line argument 0. The strings `-l`, `-t`, `abc`, `xyz`, and `bla` are called the 1st, 2nd, 3rd, 4th, and 5th command line arguments, respectively. In the above command line, there are in total 6 command line arguments (including the command name).

   How would you get the command line arguments in your progam? One way to get the command line arguments is via `argc` and `argv` parameters of the `main` function, as demonstrated in the following example:

   ```
   int main(int argc, char *argv[])
   {
      int i;

      printf("There are %d command line arguments.\n", argc);
      printf("They are:\n");
      for (i=0; i < argc; ++i)
          printf("%dth argument: %s\n", i, argv[i]);

      exit(0);
   }
   ```

   Note `argv[0]` points to the command name, `argv[1]` points to 1st command line argument, etc.

   Complete the above program and test it with different number of command line arguments.

The Weekly Lab

Submit the following exercises:

1. Theory Exercise 5.
2. C Program Exercise 2.
3. C Program Exercise 4.
4. C Program Exercise 5.

This weekly lab must be submitted to the LMS. The deadline of submission is given in the Unit Information Page of the LMS. You must adhere to the instructions in the document titled The Requirements for Documenting and Submitting Weekly Labs when preparing and submitting a weekly lab.

Dr Hong Xie
Unit Coordinator

Last update: Thursday, 7 August 2025

Document author: Dr. H. Xie, Unit Coordinator, ICT374 Operating Systems and Systems Programming
Last Modified: Thursday, 07-Aug-2025 16:07:34 AWST
Disclaimer & Copyright Notice © 2026 Murdoch University
This document is relevant for 2025 only